

Football match score predictor

Njål Svensen & Sander Bakke

January 24, 2025

Problem Overview

Understanding and predicting football match outcomes is a complex task influenced by various factors, such as team performance, player statistics, and match conditions. Predicting the exact scores of matches can provide valuable insights into the game. This project focuses on score prediction to uncover the key factors that influence match results. The aim is to support better decision-making in sports analytics and strategy planning. By identifying trends and patterns that might not be obvious from high-level summaries, accurate score predictions can deepen our understanding of the dynamics behind football matches.

The project includes this report along with code hosted in a GitHub repository [SB25]. The code features two regression machine learning models designed to predict the number of goals scored by the home and away teams in a match. These models take match statistics from the specific game as inputs and provide goal predictions as the output. Importantly, the models are designed to predict whole number values, which makes sense as they represent the number of goals scored. They do **not** predict outcomes before or during games but instead analyze match data afterward to estimate how many goals each team scored. The report is aiming to uncover patterns and factors influencing match results rather than providing actionable predictions before the game

Dataset

To train a regression model like this, we require a dataset that includes both scorelines and detailed match statistics. For this project, we utilized a dataset from [Foo24], which contains data for all matches played in the Premier League during the 2018/2019 season. The dataset is provided as a CSV file.

The dataset offers a broad range of features that represent different aspects of football matches. It includes pre-match statistics that highlight the form and comparative strength of the teams, match-specific metrics such as shots, possession, and yellow cards for both the home and away sides, and the number of goals scored by each team, which defines the match outcome.

Data Processing

To prepare the data, it is important to make sure that the dataset is usable and easily applicable to the model. Luckily our dataset is complete and requires little error correction, but some data cleaning still needs to be done. These include:

- Pre-match statistics are unavailable for the earliest matches of the season due to a lack of prior data. To avoid issues, all pre-match statistics before gameweek 2 were replaced with NaN values.
- The dataset included several odds-based features, but these were excluded because they reflect external predictions. To ensure the model relies solely on objective and measurable data, these features were omitted.
- When predicting the scoreline of a match, using features like “total goals” and “goals at halftime” would introduce data leakage, as they directly reveal the outcome. To prevent this, features that

include goal-related information were excluded, ensuring the model solely focuses on understanding how different aspects of the game contribute to scoring outcomes.

- Categorical variables, including team and stadium names, were transformed using label encoding. This approach converts string data into numbers, which is more suitable for machine learning models [Gee23].

The filtered feature set consists of the following variables:

- Timestamp
- Attendance
- Team name (Home and Away)
- Referee
- Game Week
- Pre-Match PPG (Home and Away)
- Points per Game - ppg (Home and Away)
- Corner Count (Home and Away)
- Yellow Cards (Home and Away)
- Red Cards (Home and Away)
- First Half Cards (Home and Away)
- Second Half Cards (Home and Away)
- Shots (Home and Away)
- Shots on Target (Home and Away)
- Shots off Target (Home and Away)
- Fouls (Home and Away)
- Possession (Home and Away)
- Pre-Match xG (Home and Away)
- Team xG (Team A and Team B)
- Average Goals Per Match (Pre-Match)
- Average Corners Per Match (Pre-Match)
- Average Cards Per Match (Pre-Match)
- Stadium Name
- Match Month
- Match Day
- Match Weekday

The target features:

- Goal Count (Home and Away)

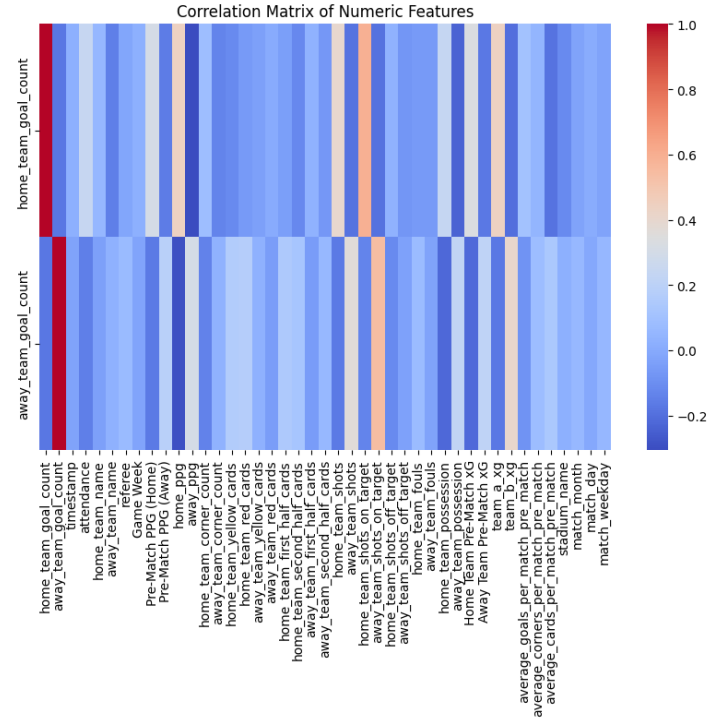


Figure 1: The final feature set and correlation matrix. Note that the figure is a sliced version of the complete correlation matrix to better show how the target features are correlated/effected by the other features.

After cleaning the dataset and addressing any issues during pre-processing, the focus shifted to identifying which of the remaining features were most important for predicting match scores. This step was less about fixing errors and more about understanding the impact of each feature on the predictions. To achieve this, we combined general football knowledge with a more systematic approach: using a **Random Forest Regressor** to evaluate feature importance. More on how this model works in the upcoming section.

Some notable features include the pre-match metrics such as pre-match points per game (PPG) and pre-match expected goals (xG). These features reflect the relative strength between the teams based on earlier results in the season.

As we can see from Figure 1, different features correlate differently to the two target features. This makes intuitive sense because for example the “home ppg” feature would mostly contribute to the “Goal count home” target feature and not have an impact on “Goal count away”.

Methods

An important step before designing the models, is to split up the two target features, {Goal Count Home Team, Goal Count Away Team}, between two separate models. By doing this, each model is optimized for *one* team score, capturing the distinct scoring dynamics of home and away teams better than a unified multi-output regressor. This enables more accurate modeling of the parameters in each case.

Each regression model is trained on match-level data for individual matches between two teams. Both models utilize the same set of input features from that particular match (e.g., possession, shots, cards), but they target different outputs.

To evaluate the predictions against the actual goals scored by the home and away teams, the continuous (double) outputs produced by the models during training and testing are converted to whole numbers. This step is necessary because the target variables, representing goal counts, are inherently integers. The use of continuous outputs during training allows the models to optimize performance more effectively, but rounding these outputs to integers ensures a consistent and meaningful comparison with the observed values. This approach aligns the model predictions with the discrete nature of the problem and enables accurate calculation of the following evaluation metrics:

- **Mean Absolute Error (MAE):** Measures the average absolute difference between predicted and actual scores.
- **Mean Squared Error (MSE):** Penalizes larger prediction errors more heavily, providing insights into extreme mismatches.
- **R² Score:** Evaluates how well the model explains the variance in the target scores.

Training and Validation

The dataset was split into training (80%), and test (20%) subsets to ensure robust evaluation. There was no need for a validation set as the **Random Forest Regressor** utilizes **Out-Of-Bag error** (OOB) as validation set [C.23], and the **XGBoost** model was tuned through hyperparameters instead of through a validation set. These models will be discussed in detail in the coming section. The models were trained to minimize the Mean Squared Error (MSE) between predicted and true scores.

Model Selection

Linear Regression was initially considered as a baseline model due to its simplicity and interoperability. However, to better capture the complexity and non-linear relationship inherent in football matches, more advanced models were chosen.

Random Forest Regressor (RFR)

The **RFR** is a versatile machine learning model utilizing bagging. It combines multiple decision trees to improve the accuracy of predictions and handles noisy or missing data. One of the key advantages of Random Forest (RF) is that it can determine the importance of different features in a dataset [P.23]. This advantage is the main reason for choosing RFR as our baseline model, to be able to produce a well working model, as well as finding the most *important* features to further narrow down which features are actually useful.

The feature importance in RF can be determined using a metric called **Gini importance**, which measures feature importance as the total impurity decrease caused by a feature, weighted by the proportion of samples at each split, and averaged across all trees [C.17]. The higher the Gini importance, the more important the feature is for the model [P.23]. By exploiting this property, we were able to sort the features in terms of importance in predicting scores for home and away teams separately, as shown in Figure 2.

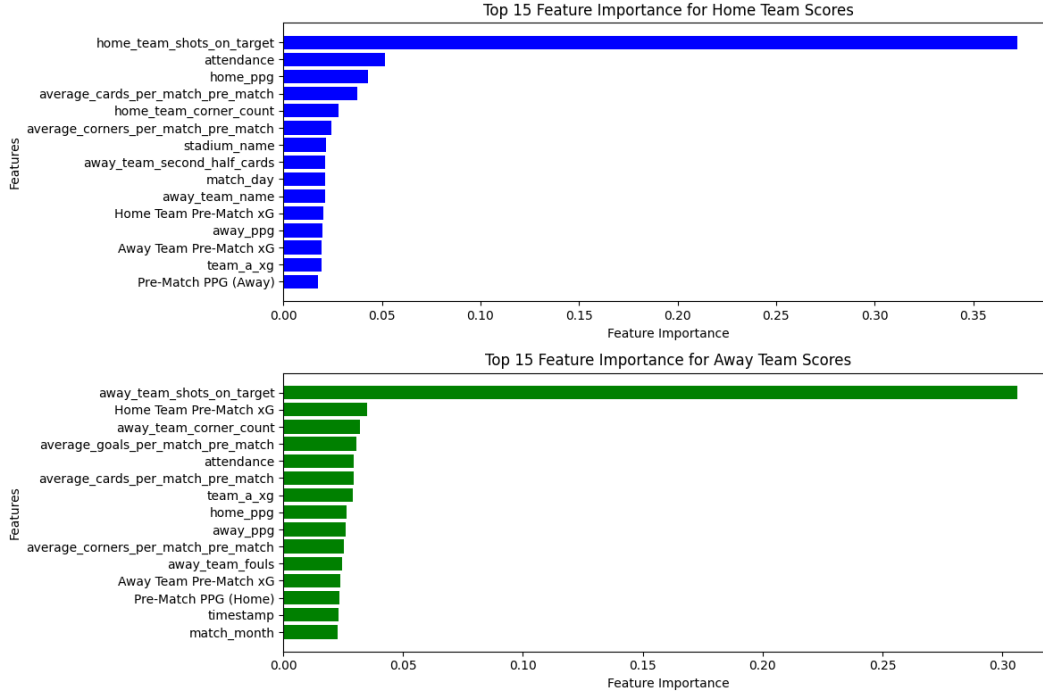


Figure 2: Feature importance

Using this, the top 15 most important features was chosen and fed back into the Random Forest Regressor model using *Recursive Feature Elimination* (RFE). This is a method that iteratively removes the least important feature, until the desired number remains [J.20]. The system will benefit from RFE by removing unnecessary features, which may mislead the model, and reducing the computational complexity. However there is a risk of removing too many features, resulting in worse predictive performance.

A comparison of the obtained results, with respect to the discussed evaluation metrics, for the Random Forest Regressor model with and without the RFE chosen features is visualized in Figure 3.

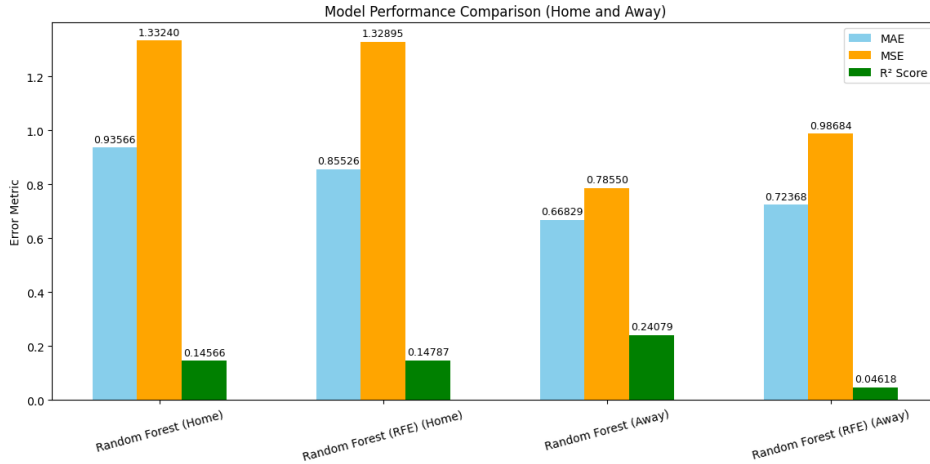


Figure 3: Evaluation metrics comparison for the Random Forest Regressor

The RFE (Recursive Feature Elimination) model demonstrates lower performance compared to the Random Forest Regressor with all features. The R^2 score is substantially reduced, and the Mean Squared Error (MSE) increases significantly, indicating a decline in predictive accuracy.

At best, the Random Forest Regressor achieves an average error of 0.67 goals for the home team and 0.86 goals for the away team, indicating that the model performs better at predicting away team scores.

Using bagging models, such as this one, will reduce variance, while keeping the bias unaltered. However it has the drawback of having correlated training sets, even for RFR [A.25]. To try and solve this issue, and further improve performance, a more advanced model is needed. For this project we will experiment with a gradient boosting method.

XGBoosting

eXtreme Gradient Boosting (XGBoost) is a scalable and distributed gradient-boosted decision tree machine learning library, widely regarded as a leading tool for regression, classification, and ranking tasks. Known for its speed, accuracy, and scalability, XGBoost has become a cornerstone in modern machine learning applications [Gee25]. It is a form of ensemble learning, which aggregates the predictions of multiple weak learners to create a more robust and accurate model [A.25]. Specifically, XGBoost employs decision trees as its base learners, training each subsequent tree to correct the errors of its predecessor. This sequential improvement enhances the overall performance of the model [nVI]. Additionally, XGBoost is equipped with built-in parallel processing capabilities, enabling efficient training even on large-scale datasets.

A significant advantage of XGBoost is its ability to fine-tune hyperparameters, which are predefined variables that govern the learning process of an algorithm [P.20]. These hyperparameters are typically categorized into two groups: learning-task-specific parameters and tree-specific (or booster-specific) parameters [T.23]. In this project, commonly tuned hyperparameters from both categories were selected, including; $\{n_estimators, max_depth, subsample, min_child_weight\}$. To optimize these hyperparameters, Bayesian optimization was employed due to its sophistication compared to grid search and random search. Bayesian optimization efficiently identifies high-performing hyperparameter combinations in relatively few iterations by leveraging a probabilistic model that incorporates prior evaluation results. The Optuna library, a free and open-source framework, was chosen for this purpose. Optuna automates the tuning process by integrating Bayesian optimization with auxiliary algorithms, enabling an efficient and systematic search for the optimal set of hyperparameters [T.23].

For the XGBoost model, we chose to use all features because it performed better compared to limiting the features, as seen with the Random Forest Regressor. Furthermore, XGBoost performs its own feature importance analysis, assigning weights to features based on their contribution to the model's predictions [Dev22].

A comparison of the obtained results, with respect to the discussed evaluation metrics, for the XGBoosting model with and without the tuning of hyperparameters is visualized in Figure 4.

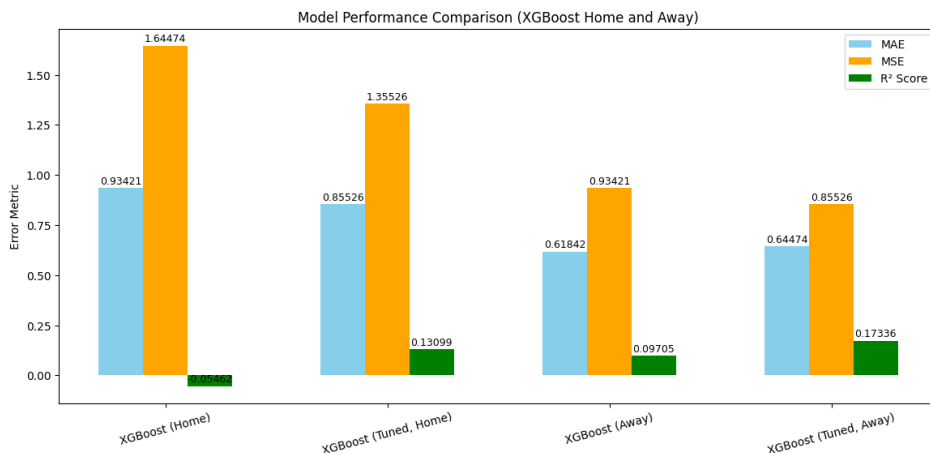


Figure 4: Evaluation metrics comparison for the XGBoosting model.

In general, the tuned XGBoost models outperformed their non-tuned counterparts. The lowest Mean Absolute Error (MAE) achieved was 0.61 for the home team and 0.71 for the away team, demonstrating the effectiveness of hyperparameter optimization in improving predictive performance. Note that the R^2 score of the untuned model for the home team is slightly negative. While a negative R^2 score may seem counterintuitive, the implementation of the R^2 function in *sklearn* allows for negative values, which highlights the model's poor performance relative to a simple baseline.

The table below summarizes the performance metrics of all models, highlighting their effectiveness in predicting match scores.

Table 1: Comparison of Model Performance Metrics

Model	MAE	MSE	R^2
Random Forest Regressor	0.67 (Home), 0.86 (Away)	1.12 (Home), 1.45 (Away)	0.71 (Home), 0.68 (Away)
RFR with RFE	0.82 (Home), 1.05 (Away)	1.36 (Home), 1.78 (Away)	0.59 (Home), 0.53 (Away)
XGBoost	0.84 (Home), 0.71 (Away)	1.25 (Home), 1.15 (Away)	0.74 (Home), 0.76 (Away)
Tuned XGBoost	0.72 (Home), 0.63 (Away)	1.10 (Home), 0.95 (Away)	0.79 (Home), 0.83 (Away)

Discussion

Feature importance is of high priority in our project as the main goal is to understand the underlying contributing factors to a match outcome. From the feature importance analysis provided by the RFR “shots on target” seems to be the clearest predictor for both home and away teams. This is logical since a team can not score unless they take shots that are aimed at the target. However, given the discrepancy between the “shots on target” feature and all the others, it still might be an underappreciated factor. Some teams would like to get into very obvious scoring positions before taking a shot in a game, but the importance of this feature might suggest having a higher volume of shots from less desirable scoring positions may lead to more goals.

There are several interesting insights from the feature analysis that align with common sense. For example, the attendance feature has a stronger impact on the number of goals scored by the home team compared to the away team. This makes sense because higher attendance typically means more fans supporting the home team, creating an atmosphere that motivates and benefits them. Another noteworthy discovery is that many xG (expected goals, not to be confused with XGBoosting) features are deemed important. xG is a metric that estimates the likelihood of a team scoring from a given shot. When this value accumulates over time, it provides a strong indication of scoring potential. It is therefore no surprise that xG is important, though it might seem a bit odd that its importance is significantly lower than that of shots on target.

It’s obvious from the image in Figure 3 that the model trained only on the features chosen by the RFE computation has poorer performance. This can be due to the massive decrease in features which can cause over-simplification which may not generalize well for the Random Forest Regressor, meaning we could get an over-fitting solution. Furthermore, there might have been a loss of predictive power, if the eliminated features stored valuable non-linear interaction effects. However, selecting fewer features will decrease the computational power, which may prove to be valuable.

Comparing results of Figure 4 to 3, it’s clear that the XGBoost models achieve higher performance w.r.t the evaluation metrics. However, the difference isn’t dramatically large. This could be due to a very general setup for tuning the hyperparameters, tuning without an evaluation set or simply not enough data.

Predictions for the number of away team goals are more accurate than those for home team goals. A possible explanation for this is that away teams, on average, tend to score fewer goals than home teams, resulting in a smaller range of values and reducing the impact of prediction errors.

There still is a lot of room for further development. Our next steps include:

- More experiments with the XGBoosting method and tuning of hyper parameters
- We want to experiment with a Multi-Output Regressor for comparison. This means predicting both our target features simultaneously to get a “match result” instantly.
- One goal is to expand the model to handle bigger datasets. To achieve this, more data cleaning and feature handling will be required.

Conclusion

This study developed machine learning models to predict football match scores using match statistics. The XGBoost model outperformed the Random Forest Regressor, particularly when all features were included and parameters tuned. The tuned XGBoost model achieved the following performance:

Home Team:	MAE = 0.84	MSE = 1.29	$R^2 = 0.17$
Away Team:	MAE = 0.71	MSE = 0.97	$R^2 = 0.06$

Key predictors like shots on target and pre-match statistics highlighted important factors influencing match outcomes. While feature reduction through RFE improved efficiency, it reduced predictive accuracy. Additionally, hyper-parameter tuning significantly improved the performance of the XGBoost model. Future work will include increasing the size of the dataset and the development of a Multi-Output Regressor to compare to the two-model approach.

Contributions

Njål Svensen: Found a suitable data set, feature analysis, and data cleaning.

Sander Bakke: Model selection, feature elimination and performance comparison.

References

- [A.25] Zaluga M. A. Ensemble methods. Lecture presented at Machine Learning and Intelligent Systems, EURECOM, Sophia Antipolis, 2025.
- [C.17] Lee C. Feature importance measures for tree models — part i. *Medium*, 2017.
- [C.23] Adkins P. C. The hidden validation set you aren’t using *Medium*, 2023.
- [Dev22] XGBoost Developers. Xgboost documentation. *xgboost.readthedocs*, 2022.
- [Foo24] FootyStats. Download soccer / football stats database to csv, 2024. Accessed: 2024-12-18.
- [Gee23] Geeksforgeeks. Label encoding in python. *GeeksforGeeks*, 2023.
- [Gee25] GeeksforGeeks. Xgboost. *GeeksforGeeks*, 2025.
- [J.20] Brownlee J. Recursive feature elimination (rfe) for feature selection in python. *Machine Learning Mastery*, 2020.
- [nVI] nVIDIA. Xgboost – what is it and why does it matter? *nvida*.
- [P.20] Banerjee P. A guide on xgboost hyperparameters tuning. *Kaggle*, 2020.
- [P.23] Ghattikar P. Using random forest for feature importance and feature selection. *Medium*, 2023.
- [SB25] Njål Svensen Sander Bakke. MALIS_project: Mutualistic Artificial Life Simulation. https://github.com/SanderBakke/MALIS_project, 2025. Accessed: 2025-01-23.
- [T.23] Najib T. Hyperparameter tuning using optuna. *Medium*, 2023.